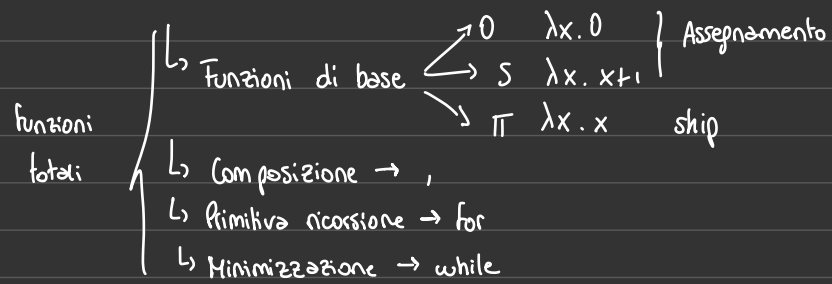
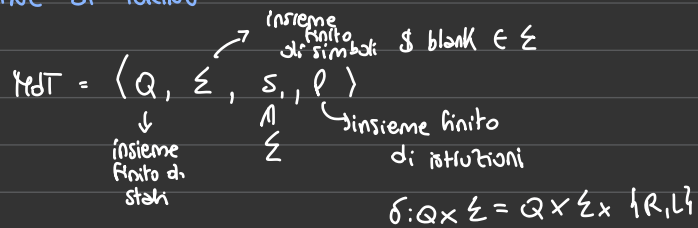


FUNZIONI RICORSIVE



MACCHINE di TURING



Th Per ogni funzione ricorsiva esiste una MdT che la calcola, e per ogni MdT esiste una funzione ricorsiva equivalente alla funzione calcolata dalla MdT.

f calcolata da una MdT $\rightarrow$ se x è sul nastro di input allora $f(x)$ si trova sul nastro alla terminazione della MdT
 $\equiv f$ Turing calcolabile se esiste una MdT che la calcola

f è ricorsiva sse è Turing calcolabile

TESI 01 CHURCH

Una funzione è intuitivamente calcolabile sse è Turing calcolabile

Aritmetizzazione delle macchine di Turing

1. Le funzioni calcolabili sono numerabili

$\Rightarrow \forall \text{MdT } M \exists n \in \mathbb{N}$ tale che M è n -esima MdT

$\hookrightarrow$ Vogliamo rappresentare univocamente ogni MdT (funzione calcolabile) con un valore naturale che identifica il suo programma / algoritmo.

$M \text{ MdT} \rightarrow n$ che lo rappresenta

$n \in \mathbb{N} \rightarrow$ posso costruire la MdT corrispondente

Q finito

$\Sigma = \{ \$, 0 \}$

$\{ L, R \}$

$\langle q, s \rangle \langle q', s', m \rangle$

Associamo ad ogni simbolo x della MdT il valore $a(x)$ dispari maggiore di 1

$q_0 \rightarrow 3$	$s_0 \rightarrow 2n+3$	$m_0 = L = 2m+2k+3$
$q_1 \rightarrow 5$	$s_1 \rightarrow 2n+5$	$m_1 = R = 2m+2k+5$
$q_2 \rightarrow 7$	$\vdots$	
$\vdots$		
$s_k \rightarrow 2n+1+2k$		
$q_n \rightarrow 2n+1$		

g_n (gödel number) associa un numero ad ogni quintupla (istruzione della MdT)

$$E = \langle qsq's'm \rangle \rightsquigarrow g_n(E) = \prod_{i=1}^k p_i^{a(x_i)}$$

$\nearrow$ prodotto
 $\nwarrow$ p_i è i -esimo numero primo

$$\begin{array}{lll}
 q_0 \rightarrow 3 & s_0 \rightarrow 9 & L \rightarrow 17 \\
 q_1 \rightarrow 5 & s_1 \rightarrow 11 & R \rightarrow 13 \\
 q_2 \rightarrow 7 & s_2 \rightarrow 13 &
 \end{array}$$

$$\begin{array}{c}
 s_3 \rightarrow 15 \\
 E = \langle q_1, s_2, q_0, s_1, L \rangle \\
 \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\
 5 \quad 13 \quad 3 \quad 11 \quad 17
 \end{array}$$

$$g_n(E) = \underbrace{3^5 \cdot 5^{13} \cdot 7^3 \cdot 11^{11} \cdot 13^{17}}_h$$

da h passo ottenere
tutti gli esponenti

$$p = \langle E_1, E_2, E_3, \dots, E_n \rangle \text{ istruzioni della MdT}$$

$$g_n(p) = \prod_{i=1}^n p_i^{g_n(E_i)}$$

$x \in \mathbb{N} \rightsquigarrow$ fattorizzazione in $\rightarrow$ insieme/numeri primi $\rightarrow$ sequenza di $g_n(\text{valori})$ che identificano le istruzioni della MdT $\langle m_1, m_2, m_3, \dots, m_n \rangle$

$\hookrightarrow$ eseguiamo fattorizzazione di ogni n_i ottenendo i valori associati ad ogni simbolo dell'istruzione $m_i = \langle m_1, m_2, m_3, m_4, m_5 \rangle$

$$\begin{array}{c}
 \mathbb{N} \\
 \psi \\
 x
 \end{array}
 \rightarrow \text{programma} \quad \varphi_x \text{ la funzione calcolata dal programma } x \in \mathbb{N}$$

$$x \in \mathbb{N} \rightsquigarrow x \text{ programma}$$

perché $\hookrightarrow$ esiste una procedura ricorsiva (algoritmo) che da x costruisce il programma (MdT / funzione ric / algoritmo) corrispondente

MdT universale $\rightsquigarrow$ interprete

È una MdT che può eseguire altre MdT

$$M_I(x_1, x_2) = M_{x_1}(x_2)$$

$\hookrightarrow$ è la MdT corrispondente all'indice x_1 .

$$x_1, x_2 \in \mathbb{N}$$

input (x_1)

input (x_2)

$M_{x_1} = \text{aritmetizzazione}^-(x_1)$ ricostruisce il programma
 M_{x_1} a partire da x_1

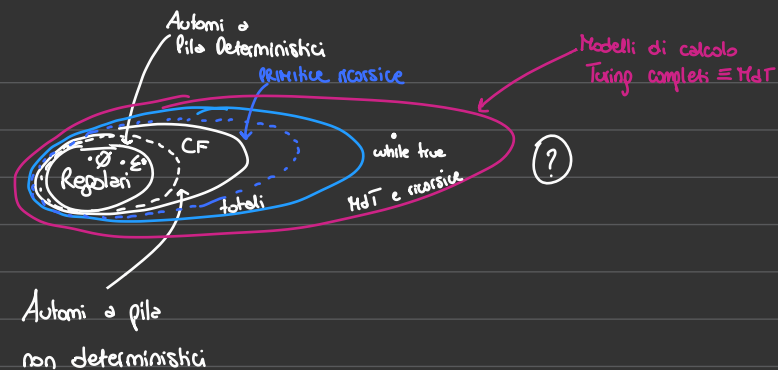
if M_{x_1} è un programma (x_1 corrisponde a una MdT) allora esegui $M_{x_1}(x_2)$

else while true

$$M_I(x_1, x_2) = \begin{cases} M_{x_1}(x_2) & \text{se } M_{x_1}(x_2) \text{ termina } \downarrow \\ \uparrow & \text{altrimenti} \end{cases}$$

$\varphi(x) \downarrow \equiv$ la funzione parziale φ applicata a x è definita e quindi termina con risultato $\varphi(x) \in \mathbb{N}$

$\varphi(x) \uparrow \equiv \varphi$ applicata a x non è definita ovvero non termina



Teorema SHN $\rightarrow$ Specializzatore

$\hookrightarrow$ Eseguire parzialmente un programma $\equiv$ specializzare l'esecuzione su una parte dell'input

$\lambda x y. x + y$

$x \rightsquigarrow 5$

abbiamo specializzato il programma sull'input x

possiamo costruire $\lambda y. 5 + y$

$\lambda x y. f(x, y)$

$f: \dots$ if $x > 0$ then $\dots$ else

$x=5$

(Th)

Per ogni coppia $m, n \geq 1$, $m, n \in \mathbb{N}$, esiste una funzione spec_m^n con $m+1$ input tale che

φ_x è una funzione in $m+n$ input, che viene specializzato su $y_1 \dots y_m$

$\varphi_x = \lambda y z. y + z. y + z$ se conosco $y = 5$

allora $\varphi_x = \lambda z. 5 + z = \varphi_{\text{spec}(x, 5)}$

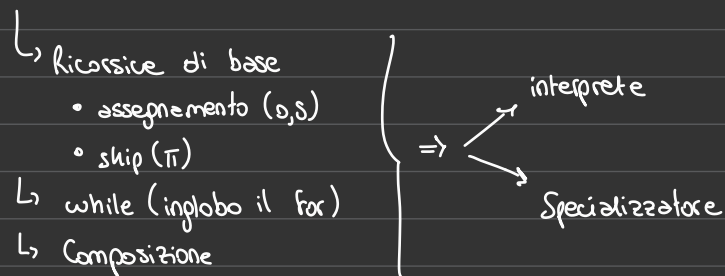
$\uparrow$

y non è più un input

Il teorema dice che il programma specializzato (ovvero in cui l'input noto è integrato nel codice) ha un codice che dipende in modo ricorsivo dal programma originale (x) e dell'input specializzato ($y_1 \dots y_m$)

$\lambda yz. y+z$		$\lambda z. S+z$
input (y)	spec	input (z)
input (z)	$\rightsquigarrow$	$y:=S;$
$w:=1$		$w:=1;$
while $w \leq y$ do $z++$; $w++$;		while $w \leq y$ do $w++$; $z++$;
output y;		output z;
φ_x		$\varphi_{spec}(x, S)$

Linguaggio Turing completo (linguaggio/modello che corrisponde allo HdT)



$\Rightarrow$ prima proiezioni di Futamura

↳ Permette di definire un compilatore usando uno specializzatore e un interprete

Dato un programma source $\in L$ esiste un programma comp che applicato a source restituisce un programma target equivalente a source.

$$\varphi_{source} = \varphi_{target} \iff \forall x. \varphi_{source}(x) = \varphi_{target}(x)$$

$$\forall x. \varphi_{source}(x) = \varphi_{int}(\overset{\text{programma}}{source}, x) = \varphi_{spec}(int, source)^{(x)}$$

$\nearrow$ interprete
 $\nwarrow$ per definizione di spec $\lambda z. \varphi_x(yz) = \lambda z. \varphi_{spec}(x, y)(z)$

definizione di interprete
 $\varphi_x(y) = \varphi_{int}(x, y)$

L essendo Turing completo contiene l'interprete int e lo specializzatore spec

$$target \stackrel{def}{=} spec(int, source) = comp \quad \text{allora} \quad \forall x. \varphi_{source}(x) = \varphi_{target}(x)$$

target è scritto nel linguaggio in cui è scritto int
 che non è necessariamente L

Problemi non risolvibili

Problema della non termination

Esiste
 $g: \mathbb{N} \rightarrow \mathbb{N}$ t.c.

$$g(x) = \begin{cases} 1 & \text{se } \varphi_x(x) \downarrow \\ 0 & \text{se } \varphi_x(x) \uparrow \end{cases}$$

Supponiamo $\exists g$ totale di questo tipo $\nearrow$
e calcolabile $\exists$ una MDT che la calcola quindi $\exists i_0. g = \varphi_{i_0}$ (programma)
costruiamo la seguente funzione

$$g'(x) = \begin{cases} \uparrow & \text{se } g(x) = 1 \\ 0 & \text{se } g(x) = 0 \end{cases}$$

$\nwarrow$ input(x)

costruiamo $\varphi_{i_0}(x)$

if $\varphi_{i_0}(x) = 1$ then while true

else return 0

($\varphi_{i_0}(x) = 0$)

$\Rightarrow g'$ è ricorsiva calcolabile

$\Rightarrow \exists i_1. \varphi_{i_1} = g'$

Calcoliamo $\varphi_{i_1}(i_1) \downarrow \Rightarrow g'(i_1) \downarrow \Rightarrow g'(i_1) = 0$

$\Rightarrow g(i_1) = 0$

$\Rightarrow \varphi_{i_1}(i_1) \uparrow \leftarrow \text{ASSURDO}$

se $\varphi_{i_1}(i_1) \uparrow \Rightarrow g'(i_1) \uparrow \Rightarrow g'(i_1) = 1$

$g(i_1) \downarrow \leftarrow \text{ASSURDO}$

$\Rightarrow \exists g$ totale t.c.

$$g(x) = \begin{cases} 1 & \varphi_x(x) \downarrow \\ 0 & \varphi_x(x) \uparrow \end{cases}$$